Project Report Section

Title :

API Mock Server as a Service (MERN-based PaaS Platform)

Project Members:

Name	PRN
Kedar Somnath Dhotre	23510090
Onkar Gunaji Rane	23510086
Aditya Rajkumar Pansare	23510080
Nikhil Suresh God	23510078

1. Project Overview

API Mock Server as a Service is a cloud-hosted Platform as a Service (PaaS) tool designed to help developers instantly generate mock REST APIs by uploading or pasting JSON data.

The platform eliminates backend dependency during frontend development by auto-generating fully functional CRUD endpoints that can be publicly accessed and tested.

This system is particularly useful for frontend developers, mobile app developers, QA testers, and hackathon teams who need ready APIs without building an actual backend.

2. Key Features

2.1 Project Management

- Create new mock API projects
- Unique project naming

- Auto-generated base API path
- View all created projects
- Delete projects

2.2 JSON Upload & Validation

- Paste raw JSON data
- Upload JSON files
- Automatic JSON validation
- Error detection for invalid schemas

2.3 Auto REST API Generation

For every collection in uploaded JSON, the system generates dynamic endpoints:

- GET all records
- GET record by ID
- POST new record
- PUT update record
- DELETE record

2.4 CRUD Simulation Engine

- Simulates database operations
- Stores mock data in MongoDB
- Allows real-time updates

2.5 Public Hosted APIs

- Shareable API URLs
- Accessible via browser/Postman
- Ready for frontend integration

2.6 Request Logging (Basic)

- Tracks API hits
- Stores method & timestamp
- Helps in debugging frontend apps

3. Backend Technology Stack

The backend is responsible for dynamic API generation, data storage, and request handling.

Technologies Used:

- **Node.js** → Server runtime environment
- **Express.js** → Web framework for API routing
- **MongoDB Atlas** → Cloud NoSQL database
- **Mongoose** → ODM for schema modeling
- **CORS** → Cross-origin request handling
- **Body-Parser / Express JSON** → Request parsing

Backend Responsibilities:

- Project creation APIs
- JSON schema storage
- Dynamic route generation
- CRUD simulation
- Request logging

4. Frontend Technology Stack

The frontend provides a developer dashboard to create and test mock APIs.

Technologies Used:

- **React.js** → Component-based UI library
- **JavaScript (ES6+)** → Logic handling
- **HTML5 & CSS3** → Structure & styling
- **Fetch API / Axios** → API communication
- **Bootstrap / Tailwind (optional)** → UI styling

Frontend Responsibilities:

- Project creation UI
- JSON input editor
- Endpoint viewer

- API testing console
- Response display panel

5. Deployment Architecture

The platform is deployed using modern cloud hosting services to simulate a real PaaS environment.

5.1 Backend Deployment — Render

- Hosts Node.js + Express server
- Provides public API base URL
- Connects with MongoDB Atlas
- Handles dynamic routing requests

Example Backend URL:

https://mockpaas.onrender.com

5.2 Frontend Deployment — Vercel

- Hosts React dashboard
- Provides global CDN delivery
- Connects to Render backend APIs
- Enables fast UI loading

Example Frontend URL:

https://mockpaas.vercel.app

6. Test Application — E-Commerce Mock API

To validate the platform, a test mock project named:

“ecommerce-api”

was created.

Collections Used:

- Users

- Products
- Orders
- Reviews

Sample Generated Endpoints:

GET /mock/ecommerce-api/users

POST /mock/ecommerce-api/products

PUT /mock/ecommerce-api/orders/:id

DELETE /mock/ecommerce-api/reviews/:id

Test Capabilities:

- User creation & retrieval
- Product inventory simulation
- Order placement testing
- Review management

This test application demonstrates real-world usability of the mock server for frontend integration and API testing.

7. Conclusion

The API Mock Server as a Service platform successfully provides a scalable developer utility that automates mock API creation and hosting.

By leveraging the MERN stack and cloud deployment tools like Render and Vercel, the system delivers a production-like API environment without requiring actual backend development.

This makes it a valuable PaaS solution for rapid prototyping, testing, and development acceleration.